Programación Orientada a Objetos
Boletín 3

El objetivo de este boletín es estudiar los mecanismos de herencia entre clases y el concepto de polimorfismo. Para ello, se creará una sencilla jerarquía de clases de películas (faltan muchos géneros o tipos de películas) sobre las que se definirán métodos y se especificarán las restricciones sobre las clases y los métodos de dicha jerarquía. Más concretamente:

- Creación de jerarquías de clases.
- Uso de métodos finales y abstractos.
- Uso de clases finales y abstractas.
- Polimorfismo de clases.
- Mecanismo de sobreescritura y reutilización de código (puntero super).

Ejercicio 1

2. Crear un proyecto denominado `B3E1` en el que se crearán tres clases: una clase `Pelicula`, en la que se representan las características principales de una película; una clase `Actor`, en la que se representan las características de un actor; y la clase `PrincipalB3E1` en la que se crearán instancias/objetos de las clases `Pelicula` y `Actor` para testear que los métodos y constructores están correctamente implementados.

3. Crear la clase que representa las características de una película.

Nombre de la clase: `Pelicula`

Atributos:

- `nombre` (`String`), es el nombre de la película.
- `anho` (`Integer`), es el año en el que se ha grabado la película.
- `duracion` (`Integer`), es la duración de la película en minutos.
- `actores` (`HashMap<String, HashMap<String,Actor>>`), es el conjunto de actores que han participado en la película, donde la clave "principal" se usa para obtener el `HashMap` cpn el conjunto de actores principales, mientras que la clave "secundario" se usa para obtener el `HashMap` con el conjunto de actores secundarios. Además, un actor secundario no puede ser un actor principal y viceversa.
- `director` (`String`), es el director de la película.
- `presupuesto` (`float`), es el presupuesto de la película.
- `recaudacion` (`float`), es la recaudación de la película.

Constructores:

Se debe implementar el siguiente constructor, en el que se deben cumplir las restricciones de valores indicadas:

- Constructor con los argumentos para inicializar los atributos `nombre`, `anho`, `duración`, `director` y `recaudacion`.

Se considerarán las siguientes restricciones:

- El atributo `anho` debe estar comprendido entre 1895 y 2022.

- El atributo `duracion` debe ser mayor de 5 minutos.
- El atributo `recaudacion` no debe ser negativo.

Getters/Setters:

Los atributos de la clase deben tener los correspondientes getters.

Los atributos `anho`, `duracion` y `recaudacion` deben tener los setters implementados, incluyendo las restricciones especificadas anteriormente.

Métodos funcionales:

- `public boolean darAlta(String rol, ArrayList<Actor> actores)`: dar de alta a un conjunto de actores. Los argumentos del método son el `rol` que han jugado los actores en la película (solamente puede tomar el valor de "secundario" o "principal") y un `ArrayList` de actores. El método devuelve un `boolean` indicando si el actor ha sido dado de alta o no.
- `public HashMap<String, Actor> actoresDirigidos(String director)`: devuelve el conjunto de actores que han sido dirigidos anteriormente (es decir, en otras películas) por el director de la película. Todos los `HashMap` deben recorrerse usando un `iterator` de claves.
- `public boolean exito()`: indica si la película ha sido o no un éxito. Se considera que una película es un éxito si la recaudación es mayor que el triple del presupuesto.
- `public Set<Actor> actoresFracaso()`: devuelve la lista de actores que han participado en otras películas que no han sido un éxito. Los `HashMap` se deben recorrer con un `iterator` de valores.
- `public void presupuesto()`: calcula el presupuesto de la película como la suma del caché de todos los actores más un porcentaje que dependerá del género de la película.
  - Para `Comedia` es un 20% de la suma del caché de todos los actores.
  - Para `Accion` es un 110% de la suma del caché de todos los actores.
  - Para `Terror` es un 70% de la suma del caché de todos los actores.
- `public boolean equals(Object object)`: donde se considera que dos películas son iguales si sus nombres son iguales.
- `public String toString()`: la representación en cadena de texto que debe seguirá la siguiente forma:

```
{
 "nombre": "<valor>",
 "anho": "<valor>",
 "duracion": "<valor>",
 "director": "<valor>",
 "recaudacion": "<valor>",
 "presupuesto": "<valor>",
 "actores": [
    { "principales": [ <nombre1>", "<nombre2>",..., "<nombreK>" ] },
    { "secundarios": [ "<nombre1>", "<nombre2>",..., "<nombreN>" ] }
 ]
}
```

donde `<valor>` es el valor del atributo correspondiente y nombre es el `<nombre>` del actor. Además, si los atributos no tienen valores (es decir, son `null`), entonces no formarán parte de la representación en cadena de texto del objeto correspondiente.

4. Crear la siguiente jerarquía de clases:

`Comedia` **es una** `Pelicula`

`ComediaRomantica` **es una** `Comedia`

`Parodia` **es una** `Comedia`

`ComediaNegra` **es una** `Comedia`

`Accion` **es una** `Pelicula`

`Robos` **es una** `Accion`

`Guerra` **es una** `Accion`

`Ficcion` **es una** `Accion`

`Thriller` **es una** `Accion`

`Terror` **es una** `Pelicula`

`TerrorGotico` **es una** `Terror`

`Zombies` **es una** `Terror`

`Psicologico` **es una** `Terror`

5. Crear la clase que representa las características de un actor.

Nombre de la clase: `Actor`

Atributos:

- `nombre (String)`, es el nombre del actor.
- `deseos` (`ArrayList< String>`), es el conjunto de directores con los que quiere trabajar el actor.
- `peliculas` (`HashMap<String, Pelicula>`), es el conjunto de películas en las que ha participado el actor.
- `cache` (`float`), es lo que cobra un actor por participar en la película.

Constructores:

Se deben implementar los siguientes constructores, en los que se deben cumplir las restricciones de valores indicadas:

- Constructor con los argumentos para inicializar los atributos `nombre`, `deseos` y `cache`.
- Constructor con los argumentos para inicializar los atributos `nombre` y `cache`.

Adicionalmente, se considerarán las siguientes restricciones:

- El atributo `cache` debe ser mayor de 0.
- El atributo `deseos` puede ser `null`, lo que significa que no tiene ningún deseo.

Getters/Setters:

Los atributos de la clase deben tener los correspondientes getters.

Los atributos `cache` y `deseos` deben tener los setters implementados, incluyendo las restricciones especificadas anteriormente.

Métodos funcionales:

- `public void introducirPelicula(Pelicula pelicula)`: introduce una `pelicula` en la que ha participado el actor.

- `public Set< Pelicula> peliculasDeGenero(String genero)`: devuelve las películas de un determinado `genero` en las que ha participado el actor.
- `public ArrayList<Pelicula> peliculasFracaso (String genero)`: devuelve las películas de un determinado `genero` en las que ha participado el actor y que han sido un fracaso.
- `public int importancia(String tipoParticipacion)`: devuelve el número de veces que el actor participado como actor "principal" o actor "secundario" en las películas en las que ha participado.
- `public float totalGanancias()`: devuelve la cantidad total que ha ganado el actor por las películas en las que ha participado. Los `HashMap` se deben recorrer con un `iterator` de `Entry.Map`.
- `public Set<String> deseosNoCumplidos()` **:** devuelve el conjunto de directores con los que el actor quiere trabajar, pero que aún no lo han dirigido en ninguna película.
- `public boolean equals(Object object)`: donde se considera que dos actores son iguales si sus nombres son iguales.
- `public String toString()`: la representación en cadena de texto que debe seguirá la siguiente forma:

```
{
 "nombre": "<nombre>",
 "cache": "<valor>",
 "deseos": [ "<nombre1>", "<nombre2>",..., "<nombreJ>" ],
 "peliculas": [ "<nombre1>", "<nombre2>",..., "<nombreM>" ]
}
```

6. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: `PrincipalB3E1`

Operaciones a realizar en la función `main()` :

- **a.** Crear las instancias del conjunto de actores indicados en el fichero Excel (B3E1_instancias.xls).
- **b.** Mostrar por pantalla las características de los actorres `actor1`, `actor2`, `actor3`, `actor4`, `actor5` y `actor6`.
- **c.** Crear las instancias del conjunto de películas indicadas en el fichero Excel (B3E1_instancias.xls).
- **d.** Mostrar por pantalla las características de las películas `pelicula1`, `pelicula2`, `pelicula3` y `pelicula4`.
- **e.** Dar de alta a los actores creados en (**b**) en todas las películas creadas en (**c**).
- **f.** Mostrar por pantalla las características de las películas `pelicula1`, `pelicula4`, `pelicula5`, `pelicula6`, `pelicula7` y `pelicula8`.
- **g.** Mostrar por pantalla las características del actor `actor1`, `actor5`, `actor7`, `actor8`, `actor9` y `actor10`.
- **h.** Mostrar las características de todas las películas en las que ha participado el `actor5`.
- **i.** Mostrar por pantalla el conjunto de actores de la `pelicula1` que han sido dirigidos por "Martin Scorsese ".
- **j.** Mostrar por pantalla el conjunto de actores de la `pelicula6` que han sido dirigidos por "Christofer Nolan".

k. Mostrar por pantalla si las películas `pelicula4`, `pelicula5`, `pelicula6` y `pelicula7` han sido o no un éxito.

l. Mostrar por pantalla los actores de la película que han participado en otras películas que no han sido un éxito.

m. Mostrar por pantalla el presupuesto de las películas `pelicula1`, `pelicula2`, `pelicula9` y `pelicula10`.

n. Mostrar por pantalla las películas de "Ficcion" en las que ha participado `actor1`, `actor2` y `actor3`.

o. Mostrar por pantalla las películas de "ComediaRomantica" en las que ha participado `actor7`, `actor8` y `actor9`.

p. Mostrar por pantalla las películas de "Piscologico" en las que ha participado `actor10`, `actor11` y `actor12`.

q. Mostrar por pantalla las películas de "Thriller" en las que ha participado `actor13`, `actor14` y `actor15`.

r. Mostrar las veces que los actores `actor1`, `actor2`, `actor3` y `actor4` han participado como actores principales.

s. Mostrar las veces que los actores `actor5`, `actor6`, `actor7` y `actor8` han participado como actores secundarios.

t. Mostrar el total de ganancias de los actores `actor10`, `actor11`, `actor12`, `actor13` y `actor14`.

u. Mostrar los deseos no cumplidos de los actores `actor15`, `actor16`, `actor17` y `actor18`.

Ejercicio 2

1. Crear un proyecto denominado `B3E2` en el que se creará una clase `Director`, en la que se representan las características principales de un director; se modificará la clase `Pelicula` para adaptarla a la nueva clase `Director`; se introducirán restricciones a la jerarquía de clases; y se creará la clase `PrincipalB3E2` en la que se crearán instancias/objetos de las clases `Pelicula` y `Director` para testear que los métodos y constructores están correctamente implementados.

2. En la clase `Pelicula` y en la jerarquía de clases:

- Introducir la clase `Director`, en vez de considerar solamente el nombre del director, modificando el constructor y todos los métodos en los que se hace uso de `Director`.
- Evitar que los últimos niveles de la jerarquía puedan tener subclases.
- Hacer que la clase `Pelicula` y las clases derivadas del primer nivel sean abstractas (`abstract`).
- Hacer que el método `exito` sea abstracto en la clase `Pelicula`.

Añadir los siguientes Atributos:

- `recaudacion` (`HashMap<String,Float>`), es la recaudación de la película en cada uno de los países en los que se ha estrenado donde la clave es el nombre de país y el dato es la recaudación que se ha obtenido en cada país.

Constructores:

- Modificar el constructor para que tenga acepte el tipo correcto para `director` y `recaudacion`.

Getters/Setters:

- El atributo `recaudacion` debe tener el setter y el getter implementado.

Añadir los siguientes Métodos:

- `public float recaudacionTotal()`: devuelve la recaudación total de la película. Este método no puede ser sobreescrito en ninguna otra clase de la jerarquía de clases. El `HashMap` se debe recorrer a partir de las claves.
- Modificar el método `presupuesto` para que sume el caché del director en el presupuesto.
- `boolean exito()`: que indica si la película ha sido un éxito o no y que calcula dicho éxito en función del tipo de película de la siguiente forma:
  - Para `Comedia`, si la recaudación total es mayor que el presupuesto.
  - Para `Accion`, si la recaudación total es un 150% mayor que el presupuesto y si la recaudación en "Estados Unidos" es un 30% mayor que el presupuesto.
  - Para `Ficcion`, si la recaudación total es un 300% mayor que el presupuesto y la recaudación en "Francia", "Italia", "Alemania" y "Reino Unido" es 20% de la suma del caché de todos los actores en cada uno de esos países.
  - Para `Terror`, si la recaudación es mayor que la suma del caché de todos los actores.

3. Crear la clase que representa las características de un director.

Nombre de la clase: `Director`

Atributos:

- `nombre (String)`, es el nombre del actor.

- `cache` (`HashMap<String,Float>`), es el caché del director en función del género de las películas en las que participa. El género se indica en el atributo especialista y debe coincidir con el género de la jerarquía de películas.
- `peliculas` (`ArrayList<Pelicula>`), es el conjunto de películas en las que ha participado el director.
- `especialista` (`Set<String>`), son los géneros de las películas en las cuales el director es un especialista.

Constructores:

Se deben implementar un constructor con los argumentos para inicializar los atributos `nombre`, `especialista` y `cache`.

Getters/Setters:

Los atributos de la clase deben tener los correspondientes getters.

Los atributos de la clase deben tener los correspondientes setters, excepto el atributo `peliculas`.

Métodos funcionales:

- `public void introducirPelicula(Pelicula pelicula)`: introduce una `pelicula` en la que ha participado el director. Si el género de la película no coincide con la lista de las películas en las cuales es un especialista, no se deberá dar de alta.
- `public int dirigirActor (String actor)`: devuelve el número de veces que un director ha dirigido a un `actor`.
- `public Set<String> generoMasCaro ()`: devuelve el conjunto de géneros en los que el director tiene un caché más alto.
- `public HashMap<String,Pelicula> peliculasMasBeneficio(float minimo)`: devuelve las películas que ha dirigido el director que han tenido una recaudación mayor que una cantidad `minimo`, donde la clave del `HashMap` es el nombre de la película y el dato la instancia.
- `public boolean equals(Object object)`: donde se considera que dos directores son iguales si sus nombres son iguales.
- `public String toString()`: la representación en cadena de texto que debe seguirá la siguiente forma:

```
{
 "nombre": "<nombre>",
 "cache": [
            { "<genero1>", "<valor1> },
            { "<genero2>", "<valor2> },
            ...
            { "<generoN>", "<valorN> }
          ],
 "peliculas": [ "<nombre1>", "<nombre2>",..., "<nombreM>" ],
 "especialista": [ "<genero1>", "<genero2>",..., "<generoP>" ]
}
```

donde si los atributos no tienen valores (es decir, son `null`), entonces no formarán parte de la representación en cadena de texto del objeto correspondiente.

4. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: `PrincipalB3E1`

Operaciones a realizar en la función `main()`:

a. Se considera que se han creado todas las instancias indicadas en el ejercicio 1.
b. Crear las instancias del conjunto de directores indicados en el fichero Excel (B3E2_instancias.xls).
c. Mostrar por pantalla las características de todos los directores creados en el apartado (**b**).
d. Introducir en las instancias creadas en el apartado (**b**) las películas en las que han participado.
e. Mostrar por pantalla las características de todos los directores creados en el apartado (**b**).
f. Mostrar las características de los actores `actor8`, `actor10` y `actor15`.
g. Mostrar los deseos no cumplidos para los actores `actor5` y `actor9`.
h. Mostrar por pantalla el presupuesto de las películas `pelicula1`, `pelicula2`, `pelicula9` y `pelicula10`.
i. Mostrar por pantalla la recaudación total de las películas `pelicula3`, `pelicula4`, `pelicula5` y `pelicula6`.
j. Mostrar por pantalla si todas las películas creadas en el ejercicio 1 han sido un éxito.
k. Mostrar el número de veces que "Michael Caine" ha sido dirigido por "Christopher Nolan".
l. Mostrar el número de veces que "Leonardo DiCaprio" ha sido dirigido por "Martin Scorsese".
m. Mostrar el género que tiene la caché más alta para los directores `director1`, `director4`, `director5` y `director7`.
n. Mostrar las películas dirigidas por los directores `director2`, `director3` y `director6` que han tenido una recaudación mayor de 80.